

AI & ASSISTED CODING

ASSIGNMENT-9.1

Name:N.Satwika

Roll no.:2303A51690

Batch no.:28

Problem 1:

Consider the following Python function:

```
def find_max(numbers):  
    return max(numbers)
```

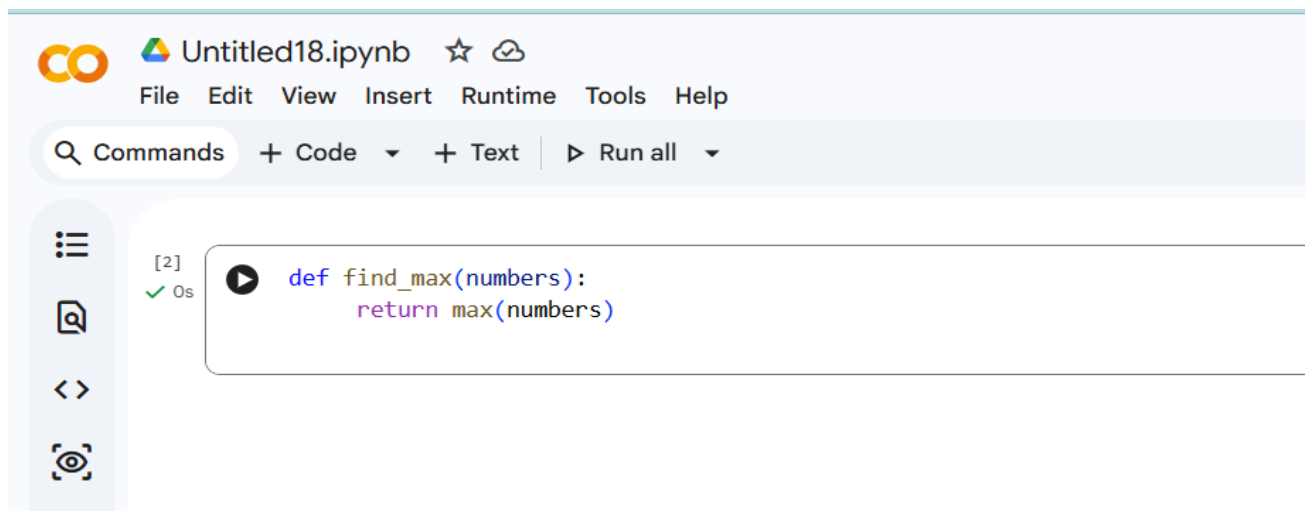
Task:

- Write documentation for the function in all three formats:
 - (a) Docstring
 - (b) Inline comments
 - (c) Google-style documentation
- Critically compare the three approaches. Discuss the advantages, disadvantages, and suitable use cases of each style.
- Recommend which documentation style is most effective for a mathematical utilities library and justify your answer.

PROMPT:

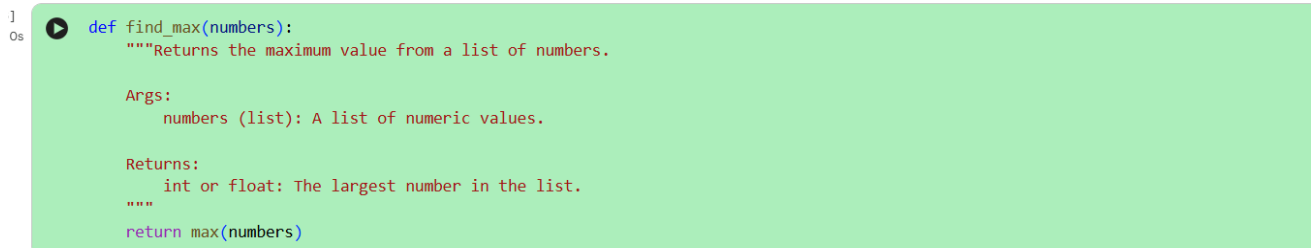
Write documentation for the given Python function using three styles: docstring, inline comments, and Google-style documentation. Then compare the styles and recommend the best one for a mathematical utilities library.

CODE AND OUTPUT:



(a) Docstring

This is a standard Python docstring, typically enclosed in triple double quotes (`"""`). It provides a concise summary of the function's purpose, arguments, and return value.



Explanation:

- Docstrings explain **what the function does**
- They are accessed using `help()` or `pydoc`
- Suitable for basic documentation

▼ (b) Inline comments

Inline comments use the hash symbol (#) to explain specific lines or blocks of code. They are generally used for explaining the 'how' rather than the 'what' of a function, and are best for complex logic or tricky parts of the code.

```
[4]
✓ Os ▶ def find_max(numbers):
        # This function takes a list of numbers as input.
        # It finds and returns the largest number in the list.
        # The built-in max() function is used for this purpose.
        return max(numbers) # Return the maximum value found in the list.
```

Explanation:

- Inline comments explain **how the code works**
- Best for complex logic
- Not useful for automatic documentation tools

▼ (c) Google-style documentation

Google-style docstrings are a popular convention that provides a more structured and detailed way to document functions, particularly useful for larger projects. They specify sections for `Args:`, `Returns:`, and `Raises:`.

```
◆ Gemini
[5]
✓ Os ▶ def find_max(numbers):
        """Returns the maximum value from a list of numbers.

        Args:
            numbers (list): The list of numeric values to search through.

        Returns:
            int or float: The largest number found in the provided list.

        Raises:
            ValueError: If the input list `numbers` is empty.
        """
        if not numbers:
            raise ValueError("Input list cannot be empty.")
        return max(numbers)
```

Explanation:

- Structured and professional format
- Clearly explains parameters and return values
- Widely used in real-world projects

Problem 2: Consider the following Python function:

```
def login(user, password, credentials):
```

```
return credentials.get(user) == password
```

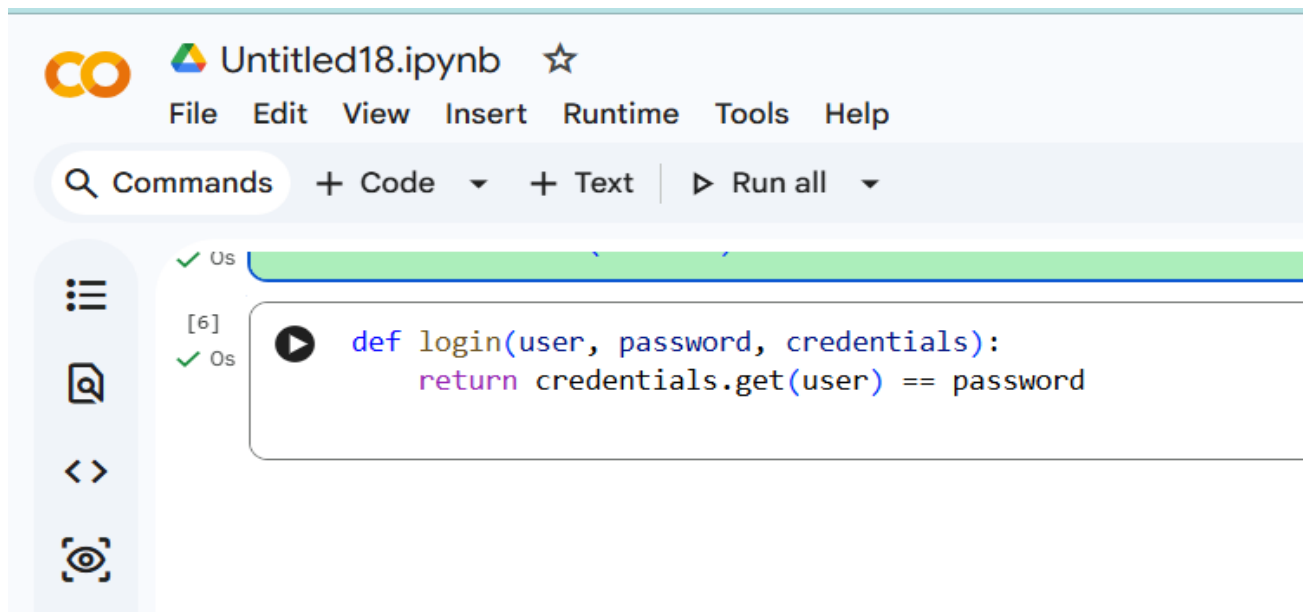
Task:

1. Write documentation in all three formats.
2. Critically compare the approaches.
3. Recommend which style would be most helpful for new developers onboarding a project, and justify your choice.

PROMPT:

Document the given login function using docstring, inline comments, and Google-style documentation. Compare the styles and recommend the best one for onboarding new developers.

CODE AND OUTPUT:



The screenshot shows a Jupyter Notebook titled "Untitled18.ipynb". The interface includes a menu bar with "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". Below the menu is a search bar labeled "Commands" and buttons for "+ Code", "+ Text", and "Run all". On the left sidebar, there are icons for a list, a search, a code editor, and a console. The main area displays a code cell with the following Python code:

```
def login(user, password, credentials):  
    return credentials.get(user) == password
```

Below the code, the output is shown as a green bar with a play button icon, indicating the code was executed successfully. The output is labeled with a green checkmark and "0s".

Docstring Documentation:

arguments, and return value as specified in the instructions.

[10]
✓ Os

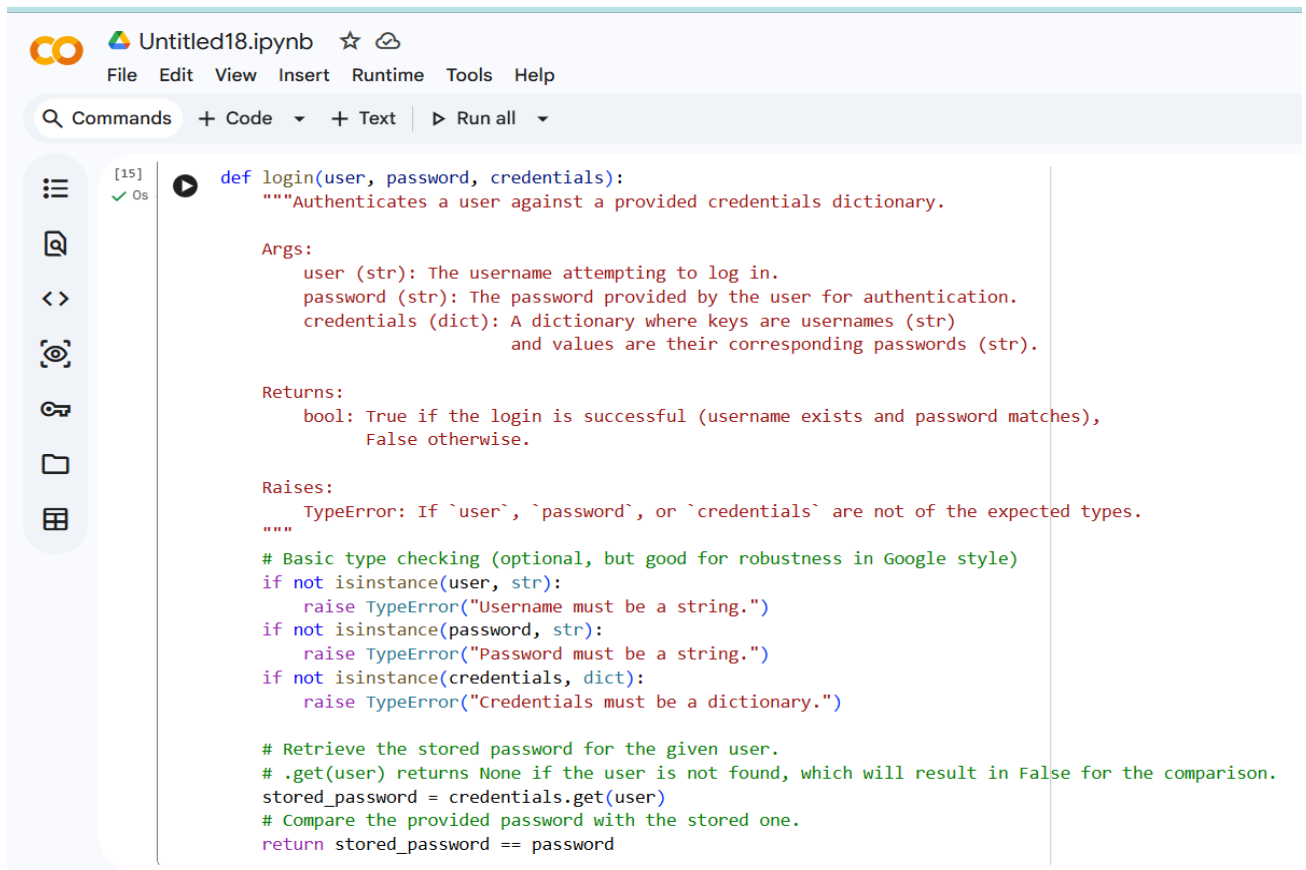
```
def login(user, password, credentials):  
    """Authenticates a user by checking credentials.  
  
    This function checks if the provided password matches the one stored  
    in a credentials dictionary for the given username.  
  
    Args:  
        user (str): The username attempting to log in.  
        password (str): The password provided by the user.  
        credentials (dict): A dictionary where keys are usernames (str)  
                           and values are their corresponding passwords (str).  
  
    Returns:  
        bool: True if the login is successful (password matches),  
              False otherwise.  
    """  
    return credentials.get(user) == password
```

(b) Inline comments:

[11]
✓ Os

```
def login(user, password, credentials):  
    # Retrieve the stored password for the given user from the credentials dictionary.  
    # If the user is not found, .get() returns None.  
    stored_password = credentials.get(user)  
  
    # Compare the retrieved stored password with the password provided by the user.  
    # This comparison determines if the login is successful.  
    return stored_password == password
```

C)Google-style Documentation



```
[15] def login(user, password, credentials):
    """Authenticates a user against a provided credentials dictionary.

    Args:
        user (str): The username attempting to log in.
        password (str): The password provided by the user for authentication.
        credentials (dict): A dictionary where keys are usernames (str)
                           and values are their corresponding passwords (str).

    Returns:
        bool: True if the login is successful (username exists and password matches),
              False otherwise.

    Raises:
        TypeError: If `user`, `password`, or `credentials` are not of the expected types.
    """
    # Basic type checking (optional, but good for robustness in Google style)
    if not isinstance(user, str):
        raise TypeError("Username must be a string.")
    if not isinstance(password, str):
        raise TypeError("Password must be a string.")
    if not isinstance(credentials, dict):
        raise TypeError("Credentials must be a dictionary.")

    # Retrieve the stored password for the given user.
    # .get(user) returns None if the user is not found, which will result in False for the comparison.
    stored_password = credentials.get(user)
    # Compare the provided password with the stored one.
    return stored_password == password
```

EXPLANATION:

- The function checks whether the entered password matches the stored credentials.
- Docstring documentation gives a basic description of the function.
- Inline comments explain the internal working of the code.
- Google-style documentation clearly defines parameters and return values.
- This style is easy for new developers to understand and use.
- Hence, Google-style documentation is the most effective for onboarding.

Problem 3: Calculator (Automatic Documentation Generation)

Task: Design a Python module named `calculator.py` and demonstrate automatic documentation generation.

Instructions:

1. Create a Python module `calculator.py` that includes the following functions, each written with appropriate docstrings:

- o `add(a, b)` – returns the sum of two numbers
 - o `subtract(a, b)` – returns the difference of two numbers
 - o `multiply(a, b)` – returns the product of two numbers
 - o `divide(a, b)` – returns the quotient of two numbers
2. Display the module documentation in the terminal using Python's documentation tools.
 3. Generate and export the module documentation in HTML format using the `pydoc` utility, and open the generated HTML file in a web browser to verify the output.

PROMPT:

Create a Python module with arithmetic functions and demonstrate automatic documentation generation using `help()` and `pydoc`.

CODE AND OUTPUT:

```
"""
Calculator Module
Provides basic arithmetic operations.
"""

def add(a, b):
    """Returns the sum of two numbers."""
    return a + b

def subtract(a, b):
    """Returns the difference of two numbers."""
    return a - b

def multiply(a, b):
    """Returns the product of two numbers."""
    return a * b

def divide(a, b):
    """Returns the quotient of two numbers."""
    if b == 0:
        return "Division by zero error"
    return a / b
```

NAME calculator

NAME
calculator

```
DESCRIPTION
    Calculator Module
    Provides basic arithmetic operations.
```

```
FUNCTIONS
    Calculator Module
    Provides basic arithmetic operations.
```

FUNCTIONS
Provides basic arithmetic operations.

```
FUNCTIONS
FUNCTIONS
    add(a, b)
        Returns the sum of two numbers.
-- More --
```

```
PS C:\Users\DELL\OneDrive\Desktop\LEARNING COURSES\AI_Assisant_Coding> & "C:/Users/DELL/OneDrive/Desktop/LEARNING COURSES/AI_Assisant_Coding/.venv/Scripts/Activate.ps1"
(.venv) PS C:\Users\DELL\OneDrive\Desktop\LEARNING COURSES\AI_Assisant_Coding> python -m pydoc -w calculator
\\

<!DOCTYPE html>
<html lang="en">
<meta charset="utf-8">
<title>Python: module calculator</title>
</head><body>

<table class="heading">
<tr class="heading-text decor">
<td class="title">&nbsp;<br><strong class="title">calculator</strong></td>
<td class="extra"><a href="#">index</a><br><a href="file:c%3A%5Cusers%5Cdell%5Conedrive%5Cdesktop%5Clearning%20courses%5Cai_assisant_coding%5Ccalculator.py">c:\users\de
    <p><span class="code">Calculator&nbsp;&nbsp;&Module<br>
Provides&nbsp;&nbsp;&basic&nbsp;&arithmetric&nbsp;&operations.</span></p>
<p>
<table class="section">
<tr class="decor functions-decor heading-text">
<td class="section-title" colspan=3>&nbsp;<br><strong class="bigsection">Functions</strong></td></tr>

<tr><td class="decor functions-decor"><span class="code">&nbsp;&nbsp;&&nbsp;&nbsp;&&nbsp;&nbsp;&&nbsp;&</span></td><td>&nbsp;</td>
<td class="singlecolumn"><dl><dt><a name="-add"><strong>add</strong></a>(a, b)</dt><dd><span class="code">Returns&nbsp;&the&nbsp;&sum&nbsp;&of&nbsp;&two&nbsp;&numbers.</span></dd></dl>
<dl><dt><a name="-divide"><strong>divide</strong></a>(a, b)</dt><dd><span class="code">Returns&nbsp;&the&nbsp;&quot;quotient&nbsp;&of&nbsp;&two&nbsp;&numbers.</span></dd></dl>
<dl><dt><a name="-multiply"><strong>multiply</strong></a>(a, b)</dt><dd><span class="code">Returns&nbsp;&the&nbsp;&product&nbsp;&of&nbsp;&two&nbsp;&numbers.</span></dd></dl>
<dl><dt><a name="-subtract"><strong>subtract</strong></a>(a, b)</dt><dd><span class="code">Returns&nbsp;&the&nbsp;&difference&nbsp;&of&nbsp;&two&nbsp;&numbers.</span></dd></dl>
</td></tr></table>
</body></html>
```

[index](#)

calculator c:\users\dell\onedrive\desktop\learning courses\ai_ assisant_coding\calculator.py

Calculator Module

Provides basic arithmetic operations.

Functions

add(a, b)

Returns the sum of two numbers.

divide(a, b)

Returns the quotient of two numbers.

multiply(a, b)

Returns the product of two numbers.

subtract(a, b)

Returns the difference of two numbers.

EXPLANATION:

- 1.Task demonstrates automatic documentation generation using Python docstrings.
 - 2.Docstrings help explain the purpose and functionality of each calculator function.
 - 3.Using pydoc reduces the need for manual documentation writing.
 - 4.Terminal-based documentation allows quick access during development.
 - 5.HTML documentation provides a clear and user-friendly presentation.
 - 6Automatic documentation ensures consistency across the module.
 - 7.Well-documented code improves readability and maintainability.
 - 8.It helps new developers understand the module easily.
 - 9.The approach follows standard Python documentation practices.
 - 10.This task highlights the importance of documentation in professional software development.
-

Problem 4: Conversion Utilities Module

Task:

1. Write a module named `conversion.py` with functions:
 - o `decimal_to_binary(n)`
 - o `binary_to_decimal(b)`
 - o `decimal_to_hexadecimal(n)`
2. Use Copilot for auto-generating docstrings.
3. Generate documentation in the terminal.
4. Export the documentation in HTML format and open it in a browser.

PROMPT:

Create a conversion utilities module with Copilot-generated docstrings and generate documentation using `pydoc`.

CODE AND OUTPUT:

```
"""
Conversion Utilities Module
"""

def decimal_to_binary(n):
    """Converts a decimal number to binary."""
    """Converts a decimal number to its binary representation as a string."""
    return bin(n)[2:]

def binary_to_decimal(b):
    """Converts a binary number to decimal."""
    """Converts a binary number (given as a string) to its decimal representation as an integer."""
    return int(b, 2)

def decimal_to_hexadecimal(n):
    """Converts a decimal number to hexadecimal."""
    """Converts a decimal number to its hexadecimal representation as a string."""
    return hex(n)[2:]

def hexadecimal_to_decimal(h):
    """Converts a hexadecimal number to decimal."""
    """Converts a hexadecimal number (given as a string) to its decimal representation as an integer."""
    return int(h, 16)
```

```
(.venv) PS C:\Users\DELL\OneDrive\Desktop\LEARNING COURSES\AI_Assisant_Coding> python -m pydoc conversion
Help on module conversion:
```

```
NAME
    conversion
```

```
FILE
    c:\users\dell\onedrive\desktop\learning courses\ai_assisant_coding\conversion.py
```

```
(.venv) PS C:\Users\DELL\OneDrive\Desktop\LEARNING COURSES\AI_Assisant_Coding> █
```

```
(.venv) PS C:\Users\DELL\OneDrive\Desktop\LEARNING COURSES\AI_Assisant_Coding> python -m pydoc -w conversion
'>>
wrote conversion.html
(.venv) PS C:\Users\DELL\OneDrive\Desktop\LEARNING COURSES\AI_Assisant_Coding> █
```

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Python: module conversion</title>
</head><body>

<table class="heading">
<tr class="heading-text decor">
<td class="title">&nbsp;<br><strong class="title">conversion</strong></td>
<td class="extra"><a href=".">index</a><br><a href="file:c%3A%5Cusers%5Cdell%5Conedrive%5Cdesktop%5Clearning%20courses%5Cai_assisant_coding%5Cconversion.py">c:\users\dell
|
</a></td></tr></table>

<p></p>

</body></html>
```

EXPLANATION:

- 1.Task demonstrates the use of automatic documentation generation for utility functions.
2. Using Copilot helps in generating accurate and consistent docstrings quickly.
3. Docstrings clearly explain the purpose and functionality of each conversion function.

4. Automatic documentation reduces manual documentation effort.
 5. Terminal-based documentation allows quick reference during development.
 6. HTML documentation provides a clear and readable presentation of the module.
 7. Well-documented code improves readability and maintainability.
 8. It helps new developers understand the conversion logic easily.
 9. Using pydoc follows standard Python documentation practices.
 10. This task highlights the importance of documentation in real-world software development.
-

Problem 5 – Course Management Module

Task:

1. Create a module `course.py` with functions:
 - o `add_course(course_id, name, credits)`
 - o `remove_course(course_id)`
 - o `get_course(course_id)`
2. Add docstrings with Copilot.
3. Generate documentation in the terminal.
4. Export the documentation in HTML format and open it in a browser.

PROMPT:

Create a course management module with functions to add, remove, and retrieve courses. Generate automatic documentation.

CODE AND OUTPUT:

```

"""
Course Management Module
"""

courses = {}

def add_course(course_id, name, credits):
    """Adds a new course to the course list."""
    #docstring suggestion: "Adds a new course to the course list with the given course ID, name, and credits."
    courses[course_id] = {
        "name": name,
        "credits": credits
    }

def remove_course(course_id):
    """Removes a course using course ID."""
    #docstring suggestion: "Removes a course from the course list using the given course ID."
    courses.pop(course_id, None)

def get_course(course_id):
    """Retrieves details of a course."""
    #docstring suggestion: "Retrieves details of a course using the given course ID. Returns None if the course does not exist."
    return courses.get(course_id)

```

- PS C:\Users\DELL\OneDrive\Desktop\LEARNING COURSES\AI_Assisant_Coding> & "C:/Users/DELL/OneDrive/Desktop/LEARNING COURSES/AI_Assisant_Coding/.venv/Scripts/Activate.ps1"
- (.venv) PS C:\Users\DELL\OneDrive\Desktop\LEARNING COURSES\AI_Assisant_Coding> python -m pydoc co rse
Help on module course:

NAME
course

FILE
c:\users\dell\onedrive\desktop\learning courses\ai_assisant_coding\course.py

❖ (.venv) PS C:\Users\DELL\OneDrive\Desktop\LEARNING COURSES\AI_Assisant_Coding> █

- PS C:\Users\DELL\OneDrive\Desktop\LEARNING COURSES\AI_Assisant_Coding> & "C:/Users/DELL/OneDrive/Desktop/LEARNING COURSES/AI_Assisant_Coding/.venv/Scripts/Activate.ps1"
- (.venv) PS C:\Users\DELL\OneDrive\Desktop\LEARNING COURSES\AI_Assisant_Coding> python -m pydoc -w co rse
- >>
wrote course.html
- ❖ (.venv) PS C:\Users\DELL\OneDrive\Desktop\LEARNING COURSES\AI_Assisant_Coding> █

```

> course.html > html
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4  <meta charset="utf-8">
5  <title>Python: module course</title>
6  </head><body>
7
8  <table class="heading">
9  <tr class="heading-text decor">
10 <td class="title">&nbsp;<br><strong class="title">course</strong></td>
11 <td class="extra"><a href=".">index</a><br><a href="file:c%3A%5Cusers%5Cdell%5Conedrive%5Cdesktop%5Clearning%20courses%5Cai_assisant_coding%5Ccourse.py">c:\users\dell\one
12 <p></p>
13
14 </body></html>

```

[index](#)

course [c:\users\dell\onedrive\desktop\learning courses\ai_assisant_coding\course.py](#)

EXPLANATION:

- 1)Using Copilot helps generate consistent and accurate docstrings efficiently.
- 2)Docstrings clearly explain the purpose of each course management function.
- 3)Automatic documentation reduces manual documentation effort.
- 4)Terminal documentation provides quick reference for developers.
- 5)HTML documentation offers a structured and readable view of the module.
- 6)Well-documented code improves readability and maintainability.
- 7)It helps new developers understand the course workflow easily.
- 8)Using pydoc follows standard Python documentation practices.
- 9)This task highlights the importance of documentation in software development.